

Opcenter™ Execution Core Technical Reference: High Performance Engine

Release 8 G.1

Notices

© Siemens 2021

This documentation and all materials related to the software are confidential and proprietary to Siemens Industry Software Inc. The contents of this document are copyrighted and cannot be reproduced in any form without the written permission of Siemens Industry Software Inc. The information contained in this document is subject to change without notice. Siemens Industry Software Inc. disclaims all liability for any consequence arising from the use of this material.

EXCEPT AS EXPRESSLY PROVIDED IN YOUR LICENSE AGREEMENT, SIEMENS INDUSTRY SOFTWARE INC. MAKES NO WARRANTIES, EXPRESSED OR IMPLIED, OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING LICENSED PROGRAMS, RELATED DOCUMENTATION, OR MARKETING MATERIALS.

A list of relevant Siemens trademarks can be found [here](#). Other trademarks belong to their respective owners.

Document History

Technical Reference: High Performance Engine

July 2021

Printed in the U.S.

Contents

About This Guide	vii
Description	vii
Audience	vii
Conventions Used	vii
For Code Samples	vii
For Specific Information Types	vii
How This Guide Is Organized	viii
Related Publications	viii
Contacting Technical Support	ix
Security Information	ix
Summary of Technical Changes	ix
Chapter 1: High Performance Engine Overview	1-1
Introduction	1-1
High Performance Engine Architecture	1-2
How Transactions are Processed	1-2
Sample Transaction	1-3
How High Performance Engine Works	1-4
Types of SQL Statements	1-4
Adding High Performance CLFs in Designer	1-5
HPE Transactions	1-5
Chapter 2: Processing High Performance CLFs	2-1
Introduction	2-1
Processing High Performance CLFs Using JSON Strings and Stored Procedures	2-2
JSON Document Example	2-2
JSON Field Definitions	2-3
Main CLF Stored Procedure and Supporting Function-Based Stored Procedures	2-3
Chapter 3: Building a Sample Transaction	3-1
Introduction	3-1
Overview	3-2
Creating the Sample Service	3-3

CDO Definition Window for the Sample Service	3-3
Field Values for the Sample CDOs	3-3
Input Fields	3-4
Additional Fields for Sample CDOs	3-4
Specifying the Main Logic	3-6
1 - Capture Current Timestamp	3-7
2 - Validate the Container Name	3-7
3 - Generate InstanceIDs	3-7
4 - Main (Parent) Container Inserts	3-8
5 - Children Containers	3-14
6 - Cross References and List Fields	3-19
7 - Completion Message	3-20
Chapter 4: High Performance Engine Transactions	4-1
Introduction	4-1
Associate (HPE)	4-2
Technical Uniqueness in Associate (HPE)	4-2
Elements in Associate (HPE)	4-2
Disassociate (HPE)	4-3
Technical Uniqueness in Disassociate (HPE)	4-3
Elements in Disassociate (HPE)	4-3
Start Two-Level (HPE)	4-4
Technical Uniqueness in Start Two-Level (HPE)	4-4
Elements in Start Two-Level (HPE)	4-4
Start Two-Level Simple (HPE)	4-5
Technical Uniqueness in Start Two-Level Simple (HPE)	4-5
Elements in Start Two-Level Simple (HPE)	4-6
Terminate Lot (HPE)	4-7
Technical Uniqueness in Terminate Lot (HPE)	4-7
Elements in Terminate Lot (HPE)	4-7
XML Sample	4-8
Opcenter Execution Core Transaction Tester in HPE	4-9

WIP Messages	4-10
E-Signature	4-11
Process Timers	4-12
Numbering Rule	4-13
Chapter 5: Debugging High Performance CLF Transactions	5-1
Introduction	5-1
Error Logging and Trace Logging	5-2
Errors	5-2
Tracing	5-2
Viewing Trace Logs	5-3
Replaying a High Performance CLF	5-4
Clearing the Error and Trace Logs	5-5
Index	Index-1

About This Guide

Description

The *Siemens Opcenter Execution Core High Performance Engine Technical Reference Guide* describes how you can use the High Performance Engine (HPE) for performing business logic on the database rather than on the application server. Using HPE enables a large number of CDOs to be inserted and updated without excessive network chatter between the application server and database.

Audience

This document is intended for developers who want the application to process business logic on the database rather than the application server, which makes the execution of transactions more efficient.

Conventions Used

For Code Samples

Code samples are provided according to the development platform supported by the product. This guide uses the following font convention to depict programming code:

```
' Delete the service
  Set Assoc = Nothing
  ServLink.RemoveService("AService")
```

Important: Siemens provides sample code as is without warranty of any kind, either express or implied, including but not limited to the implied warranties of merchantability and/or fitness for a particular purpose.

Because of the limitation of the physical page, a line of code meant to occupy one line in the program wraps to the next line in the documentation. If this occurs, the wrapped lines are indented. For example:

```
Set Assoc = Nothing ServLink.RemoveService("AService");
  Set Assoc = New ServLink.AddService
```

For Specific Information Types

This guide uses these formatting conventions.

Convention	Definition	Example
<u>Blue underlined text</u>	Indicates a hyperlink, and applies only to URLs and e-mail addresses	www.siemens.com
Bold font	Indicates icon names, buttons, field names, and specific user entries	Click Associate on the toolbar. Enter a value in the Item Caption field.

Convention	Definition	Example
<i>Italic font</i>	Indicates a document title; also indicates the result of a step in a procedure	Refer to the <i>Opcenter Execution Core Designer User Guide</i> . Click Workflow . The <i>Workflow</i> page appears within the <i>Modeling</i> page.
Important:	Indicates that the omission or result of a certain action can cause minor errors, but does not necessarily cause data loss and might require repeating a procedure or changing settings	
Note:	Indicates relevant information that reinforces an idea and is not critical to any action that you must perform	

How This Guide Is Organized

This guide is organized as follows.

Chapter / Appendix	Contents
Chapter 1, High Performance Engine Overview	Explains what the High Performance Engine is and provides an overview of how it processes transactions.
Chapter 2, Processing High Performance CLFs	Explains how High Performance CLFs and database functions are processed on the database rather than the application server.
Chapter 3, Building a Sample Transaction	Explains how you can use the High Performance CLF framework to build a transaction, and provides a detailed sample.
Chapter 4, High Performance Engine Transactions	Describes the shop floor transactions that use the High Performance Engine.
Chapter 5, Debugging High Performance CLF Transactions	Explains how error logging and trace logging facilitate development, debugging, and troubleshooting when processing High Performance transactions.

Related Publications

This guide is part of the Opcenter EX CR documentation set. Refer to these Siemens publications to supplement information found in this guide:

- *Opcenter Execution Core Designer User Guide*
- *Opcenter Execution Core Installation Guide*
- *Opcenter Execution Core Shop Floor User Guide*
- *Opcenter Execution Core System Administration Guide*

Contact your Siemens representative for the latest publications, or download them from the Siemens [Support Center](#).

Contacting Technical Support

Contact Product Support via Support Center for assistance or to report any problems. Have your SoldTo number available. Your SoldTo number is included in the file that contains your serial number.

Website:

<https://support.sw.siemens.com>

Phone:

United States or Canada: +1 (800) 955-0000 or +1 (714) 952-5444

Outside the United States and Canada: Contact your local support office.

Security Information

Siemens provides products and solutions with industrial security functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial security concept. Siemens' products and solutions constitute one element of such a concept.

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place.

For additional information on industrial security measures that may be implemented, please visit

<https://www.siemens.com/industrialsecurity>.

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are used. Use of product versions that are no longer supported, and failure to apply the latest updates may increase customer's exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Security RSS Feed under

<https://www.siemens.com/industrialsecurity>.

Summary of Technical Changes

These tables summarize the change in technical content from one revision to another.

Release 8 F.1

This table summarizes the changes in technical content for release 8 F.1.

Changes in this revision: Release 8 F.1	Revised chapter or pages
This is a new publication.	

Release 8 G.1

This table summarizes the changes in technical content for release 8 G.1.

Changes in this revision: Release 8 G.1	Revised chapter or pages
Added the Terminate Lot (HPE) transactions to Chapter 4, "High Performance Engine Transactions".	Page 4-7

Chapter 1: High Performance Engine Overview

Introduction

The High Performance Engine (HPE) is a separately licensed feature that enables you to perform business logic on the database rather than on the application server. Using HPE enables a large number of configurable data objects (CDOs) to be inserted and updated without excessive network chatter between the application server and database.

In This Chapter

This chapter contains these topics:

- High Performance Engine Architecture
- How High Performance Engine Works

High Performance Engine Architecture

The High Performance Engine enables you to perform business logic on the database rather than on the application server. This process makes the execution of transactions more efficient.

How Transactions are Processed

Every Opcenter EX CR and Opcenter EX MDD transaction generates database SQL statements .

This diagram shows an example of a transaction being performed on the application server.

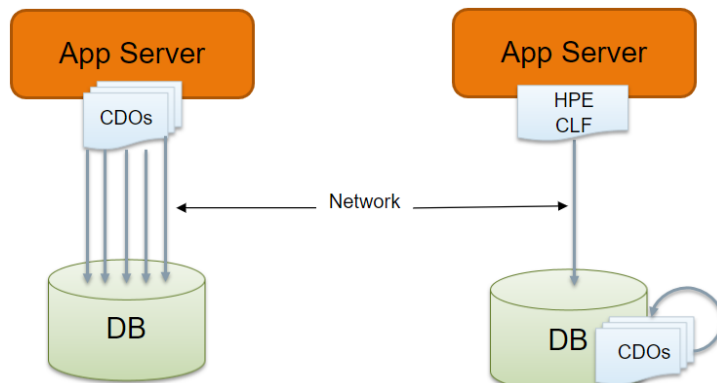


When the application performs the business logic on the application server, the SQL statements are sent to the database server one at a time, and only affect one row at a time (for example, a single INSERT or a single UPDATE statement).

A more efficient solution—especially for complex transactions—provides the following:

- One database round-trip for a set of SQL statements
- Ability to perform complex SQL statements that affect multiple rows with a single call

This solution enables a large number of CDOs to be inserted and updated without excessive network chatter between the application server and database. This diagram shows a comparison of how CDOs are processed using the two different solutions.



Sample Transaction

This image shows a sample Multi-Level Start transaction that processes 1,580,003 database records in one transaction.

Multi-Level Start (one transaction)

1. Create Main Container (1)
2. Create Child Containers (20,000)
3. Create Main Lot Attr (1)
4. Create Child Lot Attr (20,000)
5. Create Current Status (1)
6. Update Current Status (20,000)
7. Insert to StartDetailHist (20,000)
8. Insert to Traceability (1.5 M)

Total database records: 1,580,003

How High Performance Engine Works

A High Performance CLF is defined as part of a transaction and is sent to the database in the form of one or more database statements (INSERT, UPDATE, DELETE).

The application packages the High Performance CLFs and outputs them as JSON strings. It then submits the JSON strings to a stored procedure as a part of the transaction lifecycle. The stored procedure parses and executes the JSON document.

The application writes any errors it encounters when processing a High Performance CLF to an error log. It writes tracing (non-error) information to a trace log. Refer to "Debugging High Performance CLF Transactions" for information.

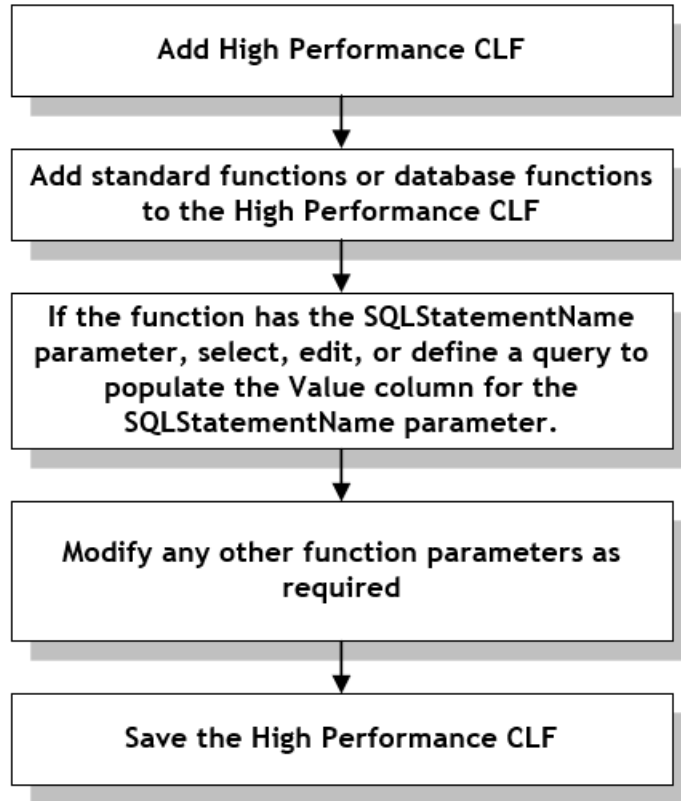
Types of SQL Statements

These are the types of SQL statements you can specify when adding database functions to a High Performance CLF:

- Simple INSERT, UPDATE, and DELETE SQL statements
- Statements with parameters (better for statement re-use)
- Repetitive statements (same statement, multiple parameter values)
- Validations (if the statement fails, the whole transaction stops)
- Returning values

Adding High Performance CLFs in Designer

You add High Performance CLFs in Designer, the application that enables you to configure Opcenter EX MDD or Opcenter EX CR to fit your factory model. This diagram shows the basic steps you must take to add a High Performance CLF in Designer.



Note: You perform High Performance development in Designer—not Designer 1.0. Designer 1.0 does not support the High Performance Engine feature.

Refer to the *Opcenter Execution Medical Device and Diagnostics Designer User Guide* or the *Opcenter Execution Core Designer User Guide* for information on adding High Performance CLFs and using database functions.

HPE Transactions

Several High Performance shop floor transactions are available if you have licensed HPE. The HPE transactions work similarly to the standard transactions. The execution of these transactions is more efficient when using the HPE version because the application processes the transaction on the database rather than the application server. These are the High Performance versions of the shop floor transactions:

- Associate (HPE)
- Disassociate (HPE)
- Start Two-Level (HPE)
- Start Two-Level Simple (HPE)

Refer to "High Performance Engine Transactions" for information.

Chapter 2: Processing High Performance CLFs

Introduction

High Performance CLFs are processed using JSON strings and stored procedures.

In This Chapter

This chapter contains this topic:

- Processing High Performance CLFs Using JSON Strings and Stored Procedures

Processing High Performance CLFs Using JSON Strings and Stored Procedures

High Performance CLFs and database functions are processed on the database rather than the application server. The application packages the High Performance CLFs and outputs them as JSON strings. It then submits the JSON strings to a stored procedure. The stored procedure parses and executes the JSON document. This process makes the execution of transactions more efficient.

JSON Document Example

Each JSON document represents one High Performance CLF. This is an example of a JSON document.

```
{
  "TransactionId": "5f4d5f910000000f",
  "CLFParameters": [
    {
      "Name": "__CDOID",
      "Value": "00170c0000000003"
    },
    {
      "Name": "__FlowVector",
      "Value": "1"
    },
    {
      "Name": "__MethodName",
      "Value": "UC01SimpleInsert"
    },
    {
      "Name": "__Return",
      "Value": ""
    }
  ],
  "Functions": [
    {
      "Name": "ExecuteSingleSQL",
      "ID": "3827",
      "Parameters": [
        {
          "Name": "SQLStatementName",
          "Value": "UC01 Simple Insert"
        }
      ]
    },
    {
      "Name": "INSTANCEID",
```

```

        "Value": "01234abcd"
      },
      {
        "Name": "NAME",
        "Value": "UseCase01"
      },
      {
        "Name": "QTY",
        "Value": "1234"
      }
    ]
  }
}

```

JSON Field Definitions

This table defines the fields in the JSON example.

Field	Definition
TransactionId	Currently executing TransactionId.
CLFParameters	Any global parameters. If a SQL statement needs a parameter value, it gets this value either locally or from this global list.
Functions	List of executions in this CLF. Each function has some identifying header information along with its list of parameter values.
ID	ID of the function. ID is used to map the function to its corresponding stored procedure.
Parameters	Name/Value pairs for parameter values.

Main CLF Stored Procedure and Supporting Function-Based Stored Procedures

High Performance CLFs call a stored procedure to process High Performance transactions. The main stored procedure parses and executes the JSON document passed to it by the application. This stored procedure also populates the JSON response package that is passed back to the High Performance CLF. There are also supporting stored procedures for each database function available in Designer.

Note: You perform High Performance development in Designer—not Designer 1.0. Designer 1.0 does not support the High Performance Engine feature.

The main stored procedure is named **clfExecute()** and all supporting stored procedures contain the **clffunc** prefix. All stored procedures are created using the script file named **DBCLFFunctions.or.sql** for Oracle. The file is installed in the following folder by default:

Camstar\InSite Administration\Scripts\Oracle

This table defines each supporting stored procedure and the function it supports.

Stored Procedure	Function	Definition
clffuncDBGenerateInstanceIDs()	DBGenerateInstanceIDs	Used by the HPE stored procedures to generate one or many Instance IDs as necessary when processing the transaction. Contains CDOTypeName, NumInstanceIDs, and Result as parameters.
clffuncDBGenerateSequence()	DBGenerateSequence	Used by the HPE stored procedures to generate a generic sequence of numbers such as 1, 2, 3, 4. Contains StartNumber, EndNumber, StepSize, and Result as parameters.
clffuncDBValidateNotExists()	DBValidateNotExists	Ensures that specific data does not already exist in the database. It executes a SQL statement that queries the database. If any rows are returned, the validation fails. Contains SQLStatementName, ErrorMessage, and ErrorID as parameters. SQLStatementName has Special Processing set to SQL Statement with Parameters. ErrorMessage defines the hard-coded string message returned if the validation fails. ErrorID defines a label name for the error message if the validation fails. Parameters for the ErrorID come from the query results. Use either ErrorMessage or ErrorID.

Stored Procedure	Function	Definition
clffuncExecuteAndFetchSQL()	ExecuteAndFetchSQL	Used by the HPE stored procedures to temporarily store variables that can be used by multiple SQL statements. The output variable contains the fetched data. Contains SQLStatementName and Result as parameters.
clffuncExecuteSingleSQL()	ExecuteSingleSQL	Takes in a set of parameters as a JSON string and executes a single SQL statement; can execute INSERT and UPDATE statements.

Note: Refer to the *Opcenter Execution Medical Device and Diagnostics Designer User Guide* or the *Opcenter Execution Core Designer User Guide* for information on adding High Performance CLFs and using database functions. Also, the Description field on the General and Parameters tabs for the selected function provides information on the intended use of the function and its parameters.

Note: CLF parameters persist across CLFs so that they can be used throughout the entire transaction. If one CLF creates database parameters (for example with the DBGenerateSequence function), the application does not delete that value until the end of the entire transaction. This enables multiple CLFs to reuse the database parameter.

Note: The application writes any errors it encounters when processing a High Performance CLF to an error log. It writes tracing (non-error) information to a trace log. There are utility stored procedures to log the errors and tracing. Refer to "Debugging High Performance CLF Transactions" for information.

Chapter 3: Building a Sample Transaction

Introduction

This section describes how to build a sample transaction. The sample transaction presented is not fully functional. It is simply an example that you can use as a guide. It demonstrates how you can use the High Performance CLF framework for a real transaction.

In This Chapter

This chapter contains these topics:

- Overview
- Creating the Sample Service
- Specifying the Main Logic

Overview

You can build a transaction from scratch—without inheriting any existing service. Use extreme care to make sure the logic you use when building the transaction matches expectations and handles various scenarios correctly.

Important: This section describes how to build a sample transaction named DBStart, but do *not* use the final product in production. This sample transaction mimics the default GroupStart transaction, having one or two levels (single container or parent + children). It is not intended to handle complex logic. The DBStart transaction built in this sample is not meant to be a replacement for the existing Start or GroupStart transactions. The sample transaction DBStart is not fully functional and does not support inheritance or extensions. It is simply an example.

Creating the Sample Service

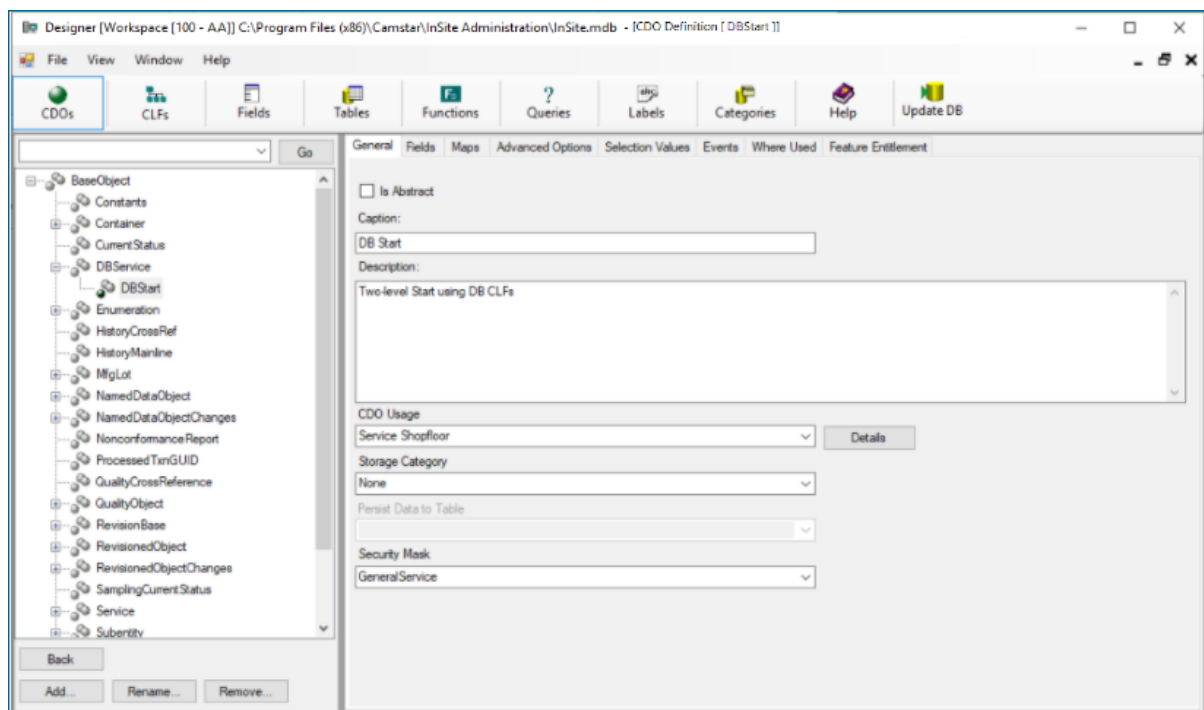
This section describes creating a sample service named DBService which contains a sample transaction named DBStart. The sample transaction built in this section is *not* a replacement for the existing Start or GroupStart transactions. It does not inherit any existing service functionality.

These are the first steps to create the sample from scratch:

- Add a new CDO under BaseObject named DBService
- Add another new CDO within DBService named DBStart

CDO Definition Window for the Sample Service

This image shows an example of the CDO Definition window with DBService and DBStart added.



Field Values for the Sample CDOs

This table lists the field values specified on the General tab for the sample CDO named DBService.

Field	Value
Is Abstract	Yes
CDO Usage	Service
Storage Category	None
Security Mask	GeneralService

This table lists the field values specified on the General tab for the sample CDO named DBStart.

Field	Value
CDO Usage	Service Shopfloor
Storage Category	None
Security Mask	GeneralService

Input Fields

This table defines the expected input fields.

Input Field	Data Type	Description
ContainerName	String	Name of the container. Must be unique.
Level	ContainerLevel	Level associated with the container being started.
Workflow	Workflow	Name of the workflow to be used for the container.
Factory	Factory	Factory where container will be started.
UOM	UOM	Unit of measure.
UOM2	UOM	Secondary unit of measure.
StartReason	StartReason	Reason for starting the container.
Product	Product	Product associated with the container.
Owner	Owner	Owner associated with the container.
MfgOrder	MfgOrder	Manufacturing order code, if any, associated with the containers being produced.
ChildContainerNameList	String	Comma-delimited list of child container names.
NumChildren	Integer	Number of child containers.

Additional Fields for Sample CDOs

In addition to the fields listed in the Input Fields table, more fields were added to DBService and DBStart. Many of them have Default Value Expressions (DVE) or Current Value Expressions (CVE), which are parsed when the field value is requested.

This table defines the additional fields added to the sample CDO named DBService.

Field	Definition	Data Type	Default Value
CompletionMsg	Message that is returned to the client. Refer to "7 – Completion Message" for information.	String	N/A
TxnDate	Transaction date.	TimeStamp	GMTToLocal(SetSystemDateGMT())
TxnDateGMT	Transaction date (GMT).	TimeStamp	SetSystemDateGMT()
User	Populated with the logged-in user.	Employee	::SessionValues.Login

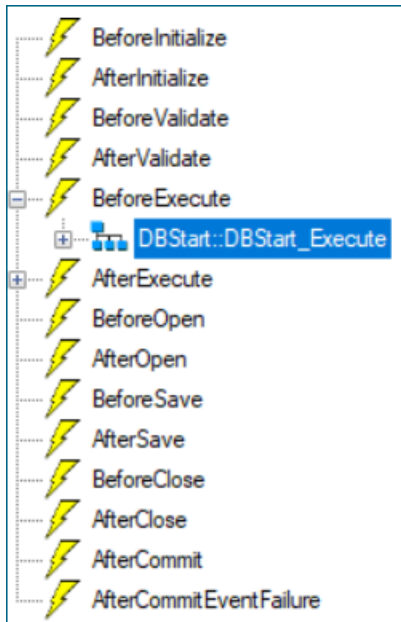
This table defines the additional field added to the sample CDO named DBStart.

Field	Definition	Data Type	Default Value
BOM	Bill of materials.	BOM	Product.BOM

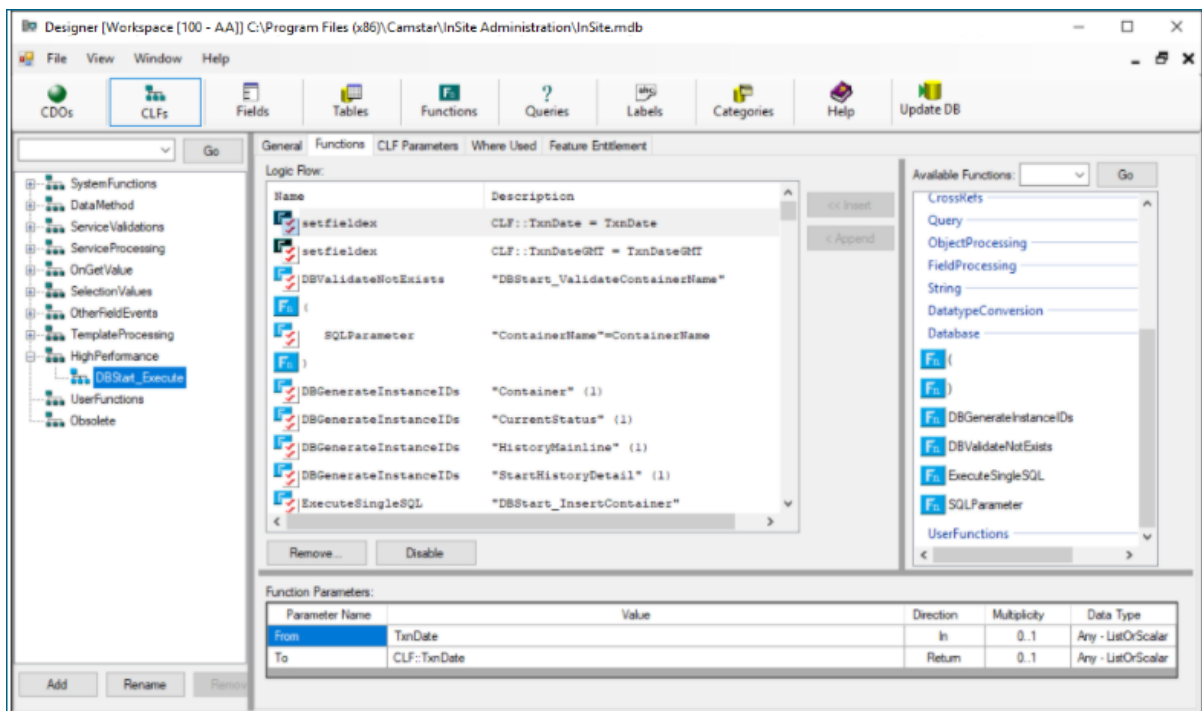
Specifying the Main Logic

All of the transaction logic is in a single High Performance CLF that is attached to the DBStart's BeforeExecute method.

This image shows an example of the method.



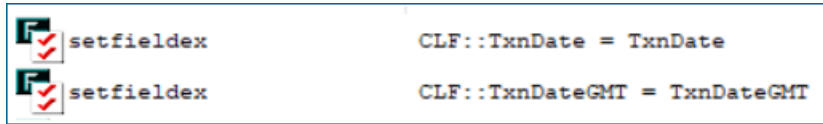
This image shows an example of a High Performance CLF named DBStart_Execute.



The steps in the following sections explain the logic used in the DBStart_Execute CLF.

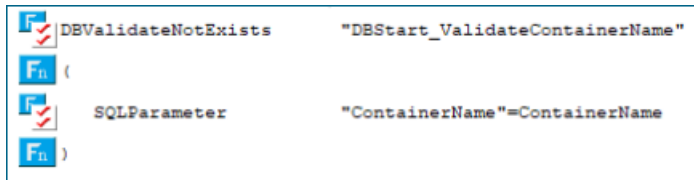
1 – Capture Current Timestamp

The current system date (and GMT) are stored in CLF:: variables so that all timestamps in the various tables match exactly. These TxnDate variables are used in several database INSERT statements.



2 – Validate the Container Name

Ensure that the parent container name is unique by executing a DBValidateNotExists function.

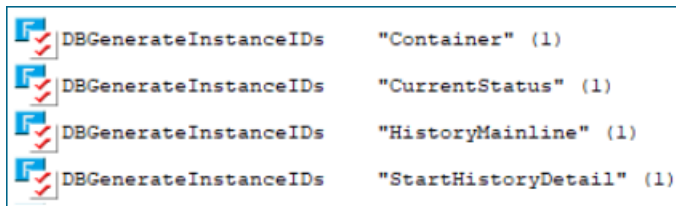


This DB function calls the following query and fails the validation if any rows are found:

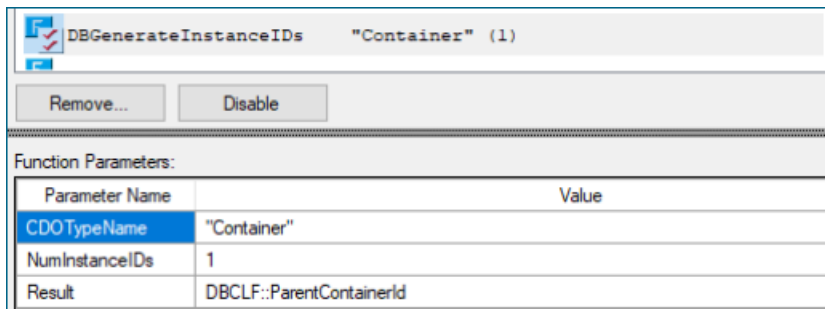
```
SELECT 'X' FROM CONTAINER WHERE ContainerName = ?ContainerName
```

3 – Generate InstanceIDs

Some InstanceIDs are used in several tables. They must be generated and stored in DBCLF:: variables. This enables those InstanceIDs to be referenced in multiple INSERT statements.



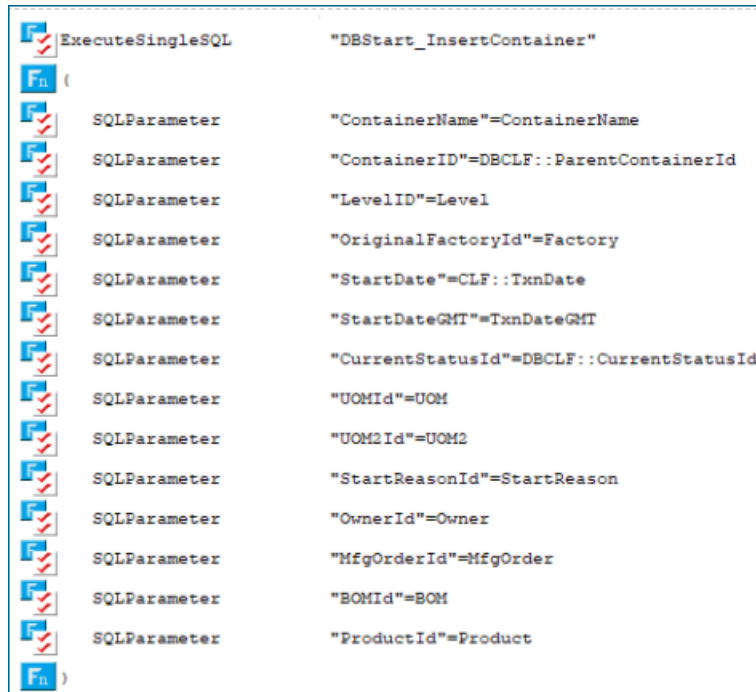
The DBGenerateInstanceIDs function generates the given number of InstanceIDs and stores them in the DBCLF:: variable passed into the function. For example, to generate one InstanceID for the Container CDO, and copy that InstanceId to DBCLF::ParentContainerId, specify the following.



4 – Main (Parent) Container Inserts

The main container will have a single row in the CONTAINER table with several other associative and history tables.

The ExecuteSingleSQL function is used for these inserts. The ExecuteSingleSQL function consists of an INSERT statement (defined in the Query Definitions tab of Designer), and several parameter mappings as shown in this example.



This image shows an example of the Query Definition.

```
INSERT INTO Container
(ContainerID, ContainerName, LevelId, OriginalFactoryId, CDOTypeID, ChangeCount, ChildCount, FactoryStartDate, FactoryStart
DateGMT, LastActivityDate, LastActivityDateGMT, OriginalContainerId, OriginalQty, OriginalStartDate, OriginalStartDateGMT
, Qty, Qty2, Status, CurrentStatusId, UOMID, UOM2ID, OriginalUOMId, OriginalUOM2Id, FactoryStartUOMId, FactoryStartUOM2Id, Sta
rtReasonId, ProductId, SamplingPassed, SamplingRequired, OwnerId, ThisContainerLost, UnitCount, MfgOrderId, InQualityContro
l, BOMId)
VALUES (?ContainerID, ?ContainerName, ?LevelID, ?OriginalFactoryId, 1040, 1, 0, ?StartDate, ?StartDateGMT, ?StartDate, ?
StartDateGMT, ?ContainerID, 1, ?StartDate, ?StartDateGMT, 15, 250, 1, ?CurrentStatusId, ?UOMId, ?UOM2Id, ?UOMId, ?UOM2Id, ?
UOMId, ?UOM2Id, ?StartReasonId, ?ProductId, 0, 0, ?OwnerId, 0, 0, ?MfgOrderId, 0, ?BOMId)
```

This INSERT statement has both dynamic values (?<parameter>) and hard-coded fixed values (CDOTypeld set to 1040).

The default String data type is appropriate for most columns. The data type must be explicitly set in the Query Definition Parameters for Timestamp columns as shown in this example.

General Details Parameters Where Used

Query Text:

```
INSERT INTO
Container (ContainerID,ContainerName,LevelId,OriginalFactoryId,CDOTypeID,ChangeCount,ChildCount,FactoryStartDate,Factor
yStartDateGMT,LastActivityDate,LastActivityDateGMT,OriginalContainerId,OriginalQty,OriginalStartDate,OriginalStartDate
GMT,Qty,Qty2,Status,CurrentStatusId,UOMID,UOM2ID,OriginalUOMId,OriginalUOM2Id,FactoryStartUOMId,FactoryStartUOM2Id,Sta
rtReasonId,ProductId,SamplingPassed,SamplingRequired,OwnerId,ThisContainerLost,UnitCount,MfgOrderId,InQualityControl,B
OMId)
VALUES (?ContainerID,?ContainerName,?LevelID,?OriginalFactoryId,1040,1,0,?StartDate,?StartDateGMT,?StartDate,?
StartDateGMT,?ContainerID,1,?StartDate,?StartDateGMT,15,250,1,?CurrentStatusId,?UOMId,?UOM2Id,?UOMId,?UOM2Id,?UOMId,?
UOM2Id,?StartReasonId,?ProductId,0,0,?OwnerId,0,0,?MfgOrderId,0,?BOMId)
```

Name	Type	Input
ContainerID	String	True
ContainerName	String	True
LevelID	String	True
OriginalFactoryId	String	True
StartDate	TimeStamp	True
StartDateGMT	TimeStamp	True
CurrentStatusId	String	True

Parameter Details

Name: StartDate

Data Type: TimeStamp

☐ List field

INSERT Statements Executed for the Main Container

This section details the various INSERT statements executed for the main container. For Object type fields (for example, Product), the application server automatically resolves the Name to its InstanceID and sends the InstanceID to the High Performance CLF. When the Source is an Object in the mappings, the actual value sent to the database is an ObjectID.

Name: DBStart_InsertContainer

SQL:

INSERT INTO Container

(ContainerID,ContainerName,LevelId,OriginalFactoryId,CDOTypeID,ChangeCount,ChildCount,FactoryStartDate,FactoryStartDateGMT,LastActivityDate,LastActivityDateGMT,OriginalContainerId,OriginalQty,OriginalStartDate,OriginalStartDateGMT,Qty,Qty2,Status,CurrentStatusId,UOMID,UOM2ID,OriginalUOMId,OriginalUOM2Id,FactoryStartUOMId,FactoryStartUOM2Id,StartReasonId,ProductId,SamplingPassed,SamplingRequired,OwnerId,ThisContainerLost,UnitCount,MfgOrderId,InQualityControl,BOMId)

VALUES

(?ContainerID,?ContainerName,?LevelID,?OriginalFactoryId,1040,1,0,?StartDate,?StartDateGMT,?StartDate,?StartDateGMT,?ContainerID,1,?StartDate,?StartDateGMT,15,250,1,?CurrentStatusId,?UOMId,?UOM2Id,?UOMId,?UOM2Id,?UOMId,?UOM2Id,?StartReasonId,?ProductId,0,0,?OwnerId,0,0,?MfgOrderId,0,?BOMId)

Mapping:

Column	Source/Value	Notes
ContainerId	DBCLF::ParentContainerId	Generated from SQL.
ContainerName	ContainerName	From input.
LevelId	Level	From input.

Column	Source/Value	Notes
OriginalFactoryId	Factory	From input.
CDOTypeId	1040	Hard-coded.
ChangeCount	1	Hard-coded.
ChildCount	NumChildren	From input.
FactoryStartDate/GMT	CLF::TxnDate/GMT	Captured at the beginning of the CLF.
LastActivityDate/GMT	CLF::TxnDate/GMT	
OriginalContainerId	DBCLF::ParentContainerId	
OriginalQty/Qty2	15/250	Sample value for this example. Should come from either input or a calculation.
Status	1	Hard-coded.
CurrentStatusId	DBCLF::CurrentStatusId	Generated from SQL.
UOMId/UOM2Id OriginalUOMId/OriginalUOM2Id FactoryStartUOMId/ FactoryStartUOM2Id	UOM/UOM2	From input.
StartReasonId	StartReason	From input.
ProductId	Product	From input.
SamplingPassed/ SamplingRequired	0	Hard-coded.
OwnerId	Owner	From input.
ThisContainerLost	0	Hard-coded.
UnitCount	NumChildren	Hard-coded.
MfgOrderId	MfgOrder	From input.
InQualityControl	0	Hard-coded.
BOMId	BOM	From BOM field, using CVE.

Name: DBStart_InsertCurrentStatus

SQL:

INSERT INTO CurrentStatus

(CurrentStatusId,CDOTypeId,ChangeCount,FactoryId,InProcess,InRework,ReworkLoopCount,ReworkTotalCount,CurrentStepPass,LastMoveDate,LastMoveDateGMT,TimersCount,WorkflowStepId,SpecId)

VALUES

(?CurrentStatusId,2140,1,?FactoryId,0,0,0,1,?LastMoveDate,?LastMoveDateGMT,0,?WorkflowStepId,?SpecId)

Mapping:

Column	Source/Value	Notes
CurrentStatusId	DBCLF::CurrentStatusId	Generated from SQL.
CDOTypeId	2140	Hard-coded.
ChangeCount	0	Hard-coded.
FactoryId	Factory	From input.
InProcess, InRework, ReworkLoopCount, ReworkTotalCount	1	Hard-coded.
CurrentStepPass	1	Hard-coded.
LastMoveDate/GMT	CLF::TxnDate	Captured at the beginning of the CLF.
TimersCount	0	Hard-coded.
WorkflowStepId	Workflow.FirstSpecStep	Workflow comes from input.
SpecId	Workflow.FirstSpecStep.Spec	

Name: DBCLF_InsertHistoryMainline

SQL:

```
INSERT INTO HistoryMainline
(CDOTypeId,HistoryMainlineId,Application,BaseTxnType,BinningIncluded,BonusIncluded,ChangeC
ount,Client,
ContainerId,DefectIncluded,EmployeeId,FactoryId,HistoryId,IconId,Implicit,InRework,LocalRe
workIncluded,
LoginId,LossIncluded,MfgDate,OperationId,OwnerId,ProductId,QualityESigConfirmed,
ReversalStatus,Status,StepPass,SystemDate,SystemDateGMT,TxnDate,TxnDateGMT,TxnId,
TxnType,UserId,WorkflowStepId,StepEntryTxnId)
```

```
VALUES (2400,?HistoryMainlineId,0,?ServiceCDOTypeId,0,0,1,0,
?ContainerId,0,?EmployeeId,?FactoryId,?HistoryId,0,0,0,0,
?EmployeeId,0,?TxnDate,?OperationId,?OwnerId,?ProductId,0,
1,1,1,?TxnDate,?TxnDateGMT,?TxnDate,?TxnDateGMT,?TxnId,
?ServiceCDOTypeId,?EmployeeId,?WorkflowStepId,?TxnId)
```

Mapping:

Column	Source/Value	Notes
HistoryMainlineId	DBCLF::HistoryMainlineId	Generated from SQL.

Column	Source/Value	Notes
ServiceCDOTypeld	CDOTypeld	CDODefID of the DBStart CDO. CDOTypeld is an inherited field from BaseObject.
ContainerId	DBCLF::ParentContainerId	Generated from SQL.
Employee	User	Populated by the system during login via ::SessionValues. (Refer to SQL.)
FactoryId	Factory	From input.
HistoryId	DBCLF::ParentContainerId	The HistoryId in HistoryMainline is generic and points to the item being updated.
TxnDate/GMT	CLF::TxnDate/GMT	
OperationId	Workflow.FirstSpecStep.Spec.Operation	
OwnerId	Owner	From input.
ProductId	Product	From input.
TxnId	DBCLF::TxnId*	Internal value*.
WorkflowStepId	Workflow.FirstSpecStep	From input, then resolved.

* The TxnId is a unique ID for each transaction and is generated by the application server during the "commit" process. Due to the timing at which the attribute gets generated, the attribute is not available during the business logic processing. Instead, it is passed directly into the High Performance CLF stored procedure. It is then stored in the database as "DBCLF::TxnId," and is available to any DB functions using this reference.

Name: DBStart_InsertParentStartHistoryDetail

SQL:

```
INSERT INTO StartHistoryDetail
(StartHistoryDetailId,HistoryMainlineId,HistoryId,CDOTypeId,ChangeCount,ChildCount,ContainerId,ExportImportKey,InQualityControl,MfgOrderId,OperationId,OwnerId,ProductId,Qty,Qty2,StartReasonId,TxnId,UnitCount,UOMId,UOM2Id,WorkflowStepId)
```

```
VALUES(clfutilNewInstanceId
('StartHistoryDetail'),?HistoryMainlineId,?HistoryId,2590,1,?ChildCount,?ContainerId,SYS_GUID
()),0,?MfgOrderId,?OperationId,?OwnerId,?ProductId,?Qty,?Qty2,?StartReasonId,?TxnId,?UnitCount,?UOMId,?UOM2Id,?WorkflowStepId)
```

Mapping:

Column	Source/Value	Notes
HistoryId	DBCLF::ParentContainerId	Refer to "HistoryId " for DBCLF_ InsertHistoryMainline.
HistoryMainlineId	DBCLF::HistoryMainlineId	Generated from SQL.
ChildCount	NumChildren	From input.
ContainerId	DBCLF::ParentContainerId	Generated from SQL.
MfgOrderId	MfgOrder	From input.
OperationId	Workflow.FirstSpecStep.Spec.Operation	
OwnerId	Owner	From input.
ProductId	Product	From input.
Qty	NumChildren	*
Qty2	0	*
StartReasonId	StartReason	From input.
TxnId	DBCLF::TxnId	Refer to * for DBCLF_ InsertHistoryMainline.
UnitCount	NumChildren	
UOMId/UOM2Id	UOM/UOM2	From input.
WorkflowStepId	Workflow.FirstSpecStep	From input.

* The Qty and Qty2 fields should eventually be summary fields for a two-level container. The Qty/Qty2 field is set on child containers, and then those fields are summed together across all children, and the total Qty/Qty2 is applied to the parent Container. This was beyond the scope of the sample presented in this topic.

Name: DBStart_InsertStepPassCount

SQL:

```
INSERT INTO StepPassCount  
(CurrentStatusId,ChangeCount,StepPassCountId,StepId,StepPass,CDOTypeId,ExportImportKey)
```

VALUES

```
(?CurrentStatusId,1,c1futilNewInstanceID('StepPassCount'),?StepId,1,7018,SYS_GUID())
```

Mapping:

Column	Source/Value	Notes
CurrentStatusId	DBCLF::CurrentStatusId	Generated from SQL.
StepId	Workflow.FirstSpecStep	From input, then resolved.

This table includes an ExportImportKey column, which is used by the export/import processes. It is a unique GUID and is not referenced anywhere else. Therefore, the Oracle function SYS_GUID() was used. This Oracle system function generates a GUID as a string, but without the spaces:

App Server Guid: 45842e27-5547-4852-ba2c-1b7f87a71096

SYS_GUID(): 45842e2755474852ba2c1b7f87a71096

Name: DBStart_InsertCurrentStatusWorkflowStack

SQL:

```
INSERT INTO CurrentStatusWorkflowStack(CurrentStatusId,FieldId,Sequence,WorkflowStackId)
VALUES (?CurrentStatusId,1415,1,?WorkflowStackId)
```

Mapping:

Column	Source/Value	Notes
CurrentStatusId	DBCLF::CurrentStatusId	Generated from SQL.
WorkflowStackId	Workflow.FirstStep	From input.

Name: DBStart_InsertHistoryCrossRef

SQL:

```
INSERT INTO HistoryCrossRef
(TrackingId,HistoryId,StartTxnId,EndTxnId,ChangeCount,HistoryCrossRefId,CrossRefType,CDOTy
peId)
VALUES (?ContainerId, ?HistoryId, ?TxnId,'7fffffffffffffff',1,clfutilNewInstanceID
('HistoryCrossRef'),0,2620)
```

Mapping:

Column	Source/Value	Notes
ContainerId	DBCLF::ParentContainerId	Generated from SQL.
HistoryId	DBCLF::ParentContainerId	
TxnId	DBCLF::TxnId	Refer to * for DBStart_ InsertHistoryMainline.

5 – Children Containers

This sample DBStart transaction is built to handle a scenario in which there are no child containers (single-level start) as well as a scenario in which there are child containers. The input field NumChildren directs this logic.

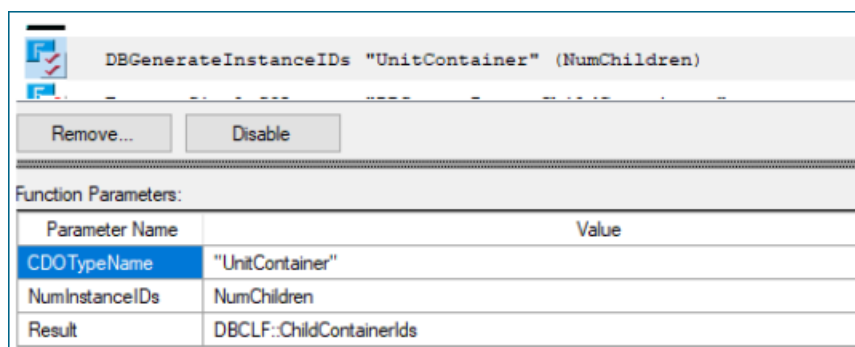
If NumChildren is zero, the application skips the child container inserts. The application includes the child container inserts if it is greater than zero. This conditional statement is handled by a traditional "if" function in Designer.

This image shows an example of the "if (NumChildren>0)" line. Its icon is green indicating that it is evaluated on the application server. Anything within the "if" block is skipped and not sent to the High Performance CLF if NumChildren is zero.



Each child container needs its own InstanceID, and those IDs are referenced in multiple inserts. The DBGenerateInstanceIDs function is used to accomplish this. The application can pass "NumChildren" as the count of InstanceIDs to generate. The output will be a comma-delimited list of InstanceIDs, and it will be stored as the database variable DBCLF::ChildContainerIds. This variable is then referenced in various INSERT statements.

This image shows an example of generating the IDs.



Name: DBStart_InsertChildContainers

SQL:

INSERT INTO Container

(ContainerID,ContainerName,ParentContainerId,LevelId,OriginalFactoryId,CDOTypeID,ChangeCount,ChildCount,FactoryStartDate,FactoryStartDateGMT,LastActivityDate,LastActivityDateGMT,OriginalContainerId,OriginalQty,OriginalStartDate,OriginalStartDateGMT,Qty,Qty2,Status,CurrentStatusId,UOMID,UOM2ID,OriginalUOMId,OriginalUOM2Id,FactoryStartUOMId,FactoryStartUOM2Id,

StartReasonId,ProductId,SamplingPassed,SamplingRequired,OwnerId,ThisContainerLost,UnitCount,MfgOrderId,InQualityControl,BOMId)

VALUES

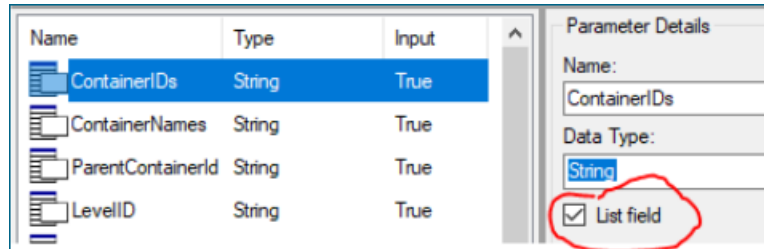
(?ContainerIDs,?ContainerNames,?ParentContainerId,?LevelID,?OriginalFactoryId,1040,1,0,?StartDate,?StartDateGMT,?StartDate,?StartDateGMT,?ContainerIDs,1,?StartDate,?StartDateGMT,15,250,1,?CurrentStatusId,?UOMId,?UOM2Id,?UOMId,?UOM2Id,?UOMId,?UOM2Id,?StartReasonId,?ProductId,0,0,?OwnerId,0,0,?MfgOrderId,0,?BOMId)

Mapping:

Column	Source/Value	Notes
ContainerId	DBCLF::ChildContainerIds	Generated from SQL; list.
ContainerName	ChildContainerNames	From input; list.
LevelId	Level	From input.
OriginalFactoryId	Factory	From input.
CDOTypeId	1040	Hard-coded.
ChangeCount	1	Hard-coded.
ChildCount	0	Hard-coded.
FactoryStartDate/GMT	CLF::TxnDate/GMT	Captured at the beginning of the CLF.
LastActivityDate/GMT	CLF::TxnDate/GMT	
OriginalContainerId	DBCLF::ParentContainerId	
OriginalQty/Qty2	15/250	Sample value for this example. Should come from either input or a calculation.
Status	1	Hard-coded.
CurrentStatusId	DBCLF::CurrentStatusId	Generated from SQL.
UOMId/UOM2Id OriginalUOMId/OriginalUOM2Id FactoryStartUOMId/ FactoryStartUOM2Id	UOM/UOM2	From input. Currently, the children are getting the parent values.
StartReasonId	StartReason	From input.
ProductId	Product	From input.
SamplingPassed/ SamplingRequired	0	Hard-coded.
OwnerId	Owner	From input.
ThisContainerLost	0	Hard-coded.
UnitCount	0	Hard-coded.
MfgOrderId	MfgOrder	From input.

Column	Source/Value	Notes
InQualityControl	0	Hard-coded.
BOMId	BOM	From BOM field, using CVE.

Note: This function inserts multiple rows—one for each child container. It is important to mark the appropriate columns as "List field" in Designer.



Only ChildContainerNames is being passed into the transaction in this example.

Name: DBStart_InsertChildrenStartHistoryDetail

SQL:

```
INSERT INTO StartHistoryDetail
(StartHistoryDetailId,HistoryMainlineId,HistoryId,CDOTypeId,ChangeCount,ChildCount,ContainerId,ExportImportKey,InQualityControl,MfgOrderId,OperationId,OwnerId,ProductId,Qty,Qty2,StartReasonId,TxnId,UnitCount,UOMId,UOM2Id,WorkflowStepId)
```

```
VALUES(clfutilNewInstanceID
('StartHistoryDetail'),?HistoryMainlineId,?HistoryId,2610,1,0,?ContainerIds,SYS_GUID
(),0,?MfgOrderId,?OperationId,?OwnerId,?ProductId,?Qty,?Qty2,?StartReasonId,?TxnId,?UnitCount,?UOMId,?UOM2Id,?WorkflowStepId)
```

Mapping:

Column	Source/Value	Notes
HistoryMainlineId	DBCLF::HistoryMainlineId	Generated from SQL; list.
HistoryId	DBCLF::ParentContainerId	
ContainerIds	DBCLF::ChildContainerIds	Generated from SQL.
MfgOrderId	MfgOrder	From input.
OperationId	Workflow.FirstSpecStep.Spec.Operation	
OwnerId	Owner	From input.
ProductId	Product	From input.
Qty	1	Hard-coded currently.
Qty2	1	Hard-coded currently.
StartReasonId	StartReason	From input.

Column	Source/Value	Notes
TxnId	DBCLF::TxnId	Refer to * for DBCLF_InsertHistoryMainline.
UnitCount	0	Hard-coded since these are child containers.
UOMId/UOM2Id	UOM/UOM2	From input. Using the parent's values for this example.
WorkflowStepId	Workflow.FirstSpecStep	From input, then resolved.

Note: The next two mappings are for the child HistoryCrossRefs. This table holds two rows for each child—one for itself and one for the parent. Note that the same Query definition is being used for the children and the parent container (above). The QueryDef has the ContainerId and HistoryId parameters defined as "list." The stored procedures are able to recognize if the input data is actually a list. For the parent container (above), the stored procedures process the insert as a single-row insert. For the children (below), it processes them as array inserts.

Name: DBStart_InsertHistoryCrossRef (child-to-self)

SQL:

```
INSERT INTO HistoryCrossRef
(TrackingId,HistoryId,StartTxnId,EndTxnId,ChangeCount,HistoryCrossRefId,CrossRefType,CDOTypeId)
```

```
VALUES (?ContainerId, ?HistoryId, ?TxnId,'7fffffffffffffff',1,clfutilNewInstanceID
('HistoryCrossRef'),0,2620)
```

Mapping:

Column	Source/Value	Notes
ContainerId	DBCLF::ChildContainerIds	Generated from SQL; list.
HistoryId	DBCLF::ChildContainerIds	Generated from SQL; list.
TxnId	DBCLF::TxnId	Refer to * for DBCLF_InsertHistoryMainline.

Name: DBStart_InsertHistoryCrossRef (child-to-parent)

SQL:

```
INSERT INTO HistoryCrossRef
(TrackingId,HistoryId,StartTxnId,EndTxnId,ChangeCount,HistoryCrossRefId,CrossRefType,CDOTypeId)
```

```
VALUES (?ContainerId, ?HistoryId, ?TxnId,'7fffffffffffffff',1,clfutilNewInstanceID
('HistoryCrossRef'),0,2620)
```


Mapping:

Column	Source/Value	Notes
ContainerId	DBCLF::ChildContainerIds	Generated from SQL; list.
HistoryId	DBCLF::ParentContainerId	
TxnId	DBCLF::TxnId	Refer to * for DBCLF_ InsertHistoryMainline.

Note: In the second insert, only the DBCLF::ChildContainerIds is a list. The DBCLF::ParentContainerId is not a list. The stored procedures handle this by using the same ParentContainerId for all rows.

6 – Cross References and List Fields

The ContainerCurrentCrossRefs is a list field, which results in a database many-to-many relationship. The necessary information is already inserted for this table. A SELECT . . . INTO type of statement is all that is needed.

Name: DBStart_InsertContainerCurrentCrossRef

SQL:

```
INSERT INTO ContainerCurrentCrossRefs (ContainerId,FieldId,Sequence,CurrentCrossRefsId)
SELECT TrackingId ContainerId,2072 FieldId,CASE TrackingId WHEN HistoryId THEN 1 ELSE 2
END Seq, HistoryCrossRefId CurrentCrossRefsId
FROM HistoryCrossRef
WHERE StartTxnId = ?TxnId
```

Mapping:

Column	Source/Value	Notes
TxnId	DBCLF::TxnId	Refer to * for DBCLF_ InsertHistoryMainline.

Note: The structure of this SQL statement enables the use of already inserted data to build the data structure that ContainerCurrentCorssRefs needs. FieldId is hard-coded in this example to 2072. This is a metadata value that should not change.

Name: DBStart_InsertHistoryMainlineHistoryDetails

SQL:

```
INSERT INTO HistoryMainlineHistoryDetails
(HistoryMainlineId,FieldId,Sequence,HistoryDetailsId)
SELECT HistoryMainlineId,1968,rownum,StartHistoryDetailId
FROM StartHistoryDetail
WHERE TxnId = ?TxnId
```

Mapping:

Column	Source/Value	Notes
TxnId	DBCLF::TxnId	Refer to * for DBCLF_ InsertHistoryMainline.

Note: The structure of this SQL statement enables the use of data that has already been inserted into other tables. The data just needs to be structured for this table. However, this INSERT statement needs a Sequence which increments for each row. The Oracle ROWNUM built-in function handles this—it increments for each row. FieldId is hard-coded to 1968.

Name: DBStart_InsertStartHistoryDtlWorkflowStack

SQL:

```
INSERT INTO StartHistoryDtlWorkflowStack
(StartHistoryDetailId,FieldId,Sequence,WorkflowStackId)
SELECT StartHistoryDetailId,5546,1,?WorkflowStackId
FROM StartHistoryDetail
WHERE TxnId = ?TxnId
```

Mapping:

Column	Source/Value	Notes
TxnId	DBCLF::TxnId	Refer to * for DBCLF_ InsertHistoryMainline.
WorkflowStackId	Workflow.FirstStep	

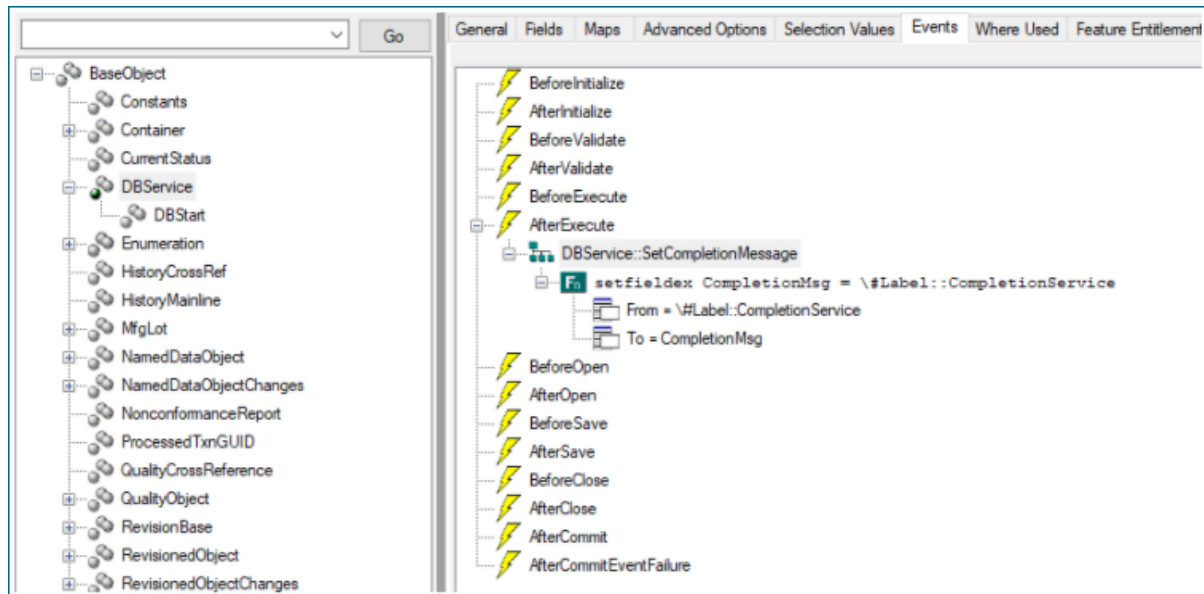
Note: FieldId is hard-coded to 5546.

7 – Completion Message

Services typically have a CompletionMsg field that is returned and displayed to the user. A CompletionMsg field was added to the base DBService CDO.

To support localization, default services use labels to format the CompletionMsg field as a part of the AfterExecute event. A generic message is populated at the top-level service CDO.

This image shows an example of attaching the SetCompletionMessage CLF to the AfterExecute event. The message references the CompletionService Label.

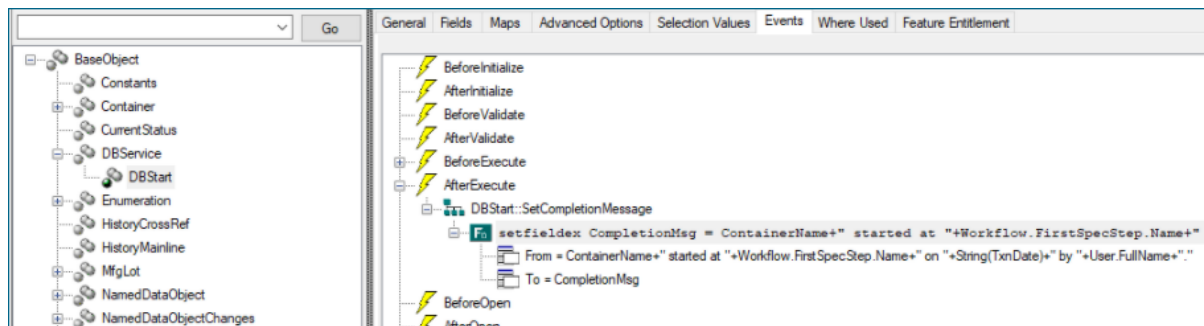


For this DBStart example, the message needs to be:

"ContainerABC" started at Step 1 on 02/04/2021 5:30:22 PM by Administrator.

The Default "CompletionStart" Label does not apply because this example does not use existing service CDOs such as Start, StartDetails, and CurrentStatusDetails.

This image shows an example of building the string in the correct (English) format in the AfterExecute Event and assigning that string to CompletionMsg.



Chapter 4: High Performance Engine Transactions

Introduction

This section describes the shop floor transactions that use the High Performance Engine (HPE). The HPE versions of these transactions are available if you have licensed HPE.

The HPE transactions work similarly to the standard transactions. However, the execution of these transactions is more efficient when using the HPE versions as the application processes the HPE transaction on the database rather than the application server.

Refer to the *Opcenter Execution Medical Device and Diagnostics Shop Floor User Guide* or the *Opcenter Execution Core Shop Floor User Guide* for information on the standard shop floor transactions.

Note: ProcessLabelInformation is not supported in HPE for Release 8.5.

In This Chapter

This chapter contains these topics:

- Associate (HPE)
- Disassociate (HPE)
- Start Two-Level (HPE)
- Start Two-Level Simple (HPE)
- Terminate Lot (HPE)
- XML Sample
- Opcenter Execution Core Transaction Tester in HPE
- WIP Messages
- E-Signature
- Process Timers
- Numbering Rule

Associate (HPE)

The page for the Associate (HPE) transaction is the same as the page for the standard Associate transaction. This transaction creates parent and child container relationships. It enables you to group existing containers under a single parent. Any action performed once on the parent container applies to the child containers automatically.

Refer to the *Opcenter Execution Medical Device and Diagnostics Shop Floor User Guide* or the *Opcenter Execution Core Shop Floor User Guide* for information on Associate.

Note: Associate (HPE) does not support transaction reversal.

Technical Uniqueness in Associate (HPE)

The standard Associate transaction makes round-trips from the application server to the database.

For example, when you create a Container with five child containers, a total of six INSERT statements are sent to the database one at a time. With HPE, the SQL statements are sent to the database all at once.

SQL statements, CLFs DBAssociate_Initialize, DBAssociate_Validate and DBAssociate_Execute are created specifically for the Associate (HPE) transaction under the High Performance CLF in the Designer.

Elements in Associate (HPE)

CDO

A new CDO called DBAssociate is created as a subclass of the Associate CDO. DBAssociate_Initialize, DBAssociate_Validate and DBAssociate_Execute are attached to the Events.

Fields

These fields are related to DBAssociate:

- ChildContainers list object is inherited from Associate CDO.
- ChildCount is created and it is the total number of child containers.
- ContainerNamesStr is created and it is in the form of long string, containing all the child containers' names. The names of the child containers are separated by the delimiter "|".
- IsDBTxn is a flag that determines if the service belongs to HPE transaction. It is inherited from ContainerTxn CDO.

CLFs

- DBAssociate_Execute - Contains all the INSERT/UPDATE/DELETE statements that were previously created within application server.
- DBAssociate_Initialize - Creates temporary tables in the database. This CLF is used to store necessary data to be used as parameter for INSERT/UPDATE/DELETE statements in DBAssociate_Validate and DBAssociate_Execute and other DBCLF.
- DBAssociate_Validate - Performs validation for container and child containers.

Disassociate (HPE)

The page for the Disassociate (HPE) transaction is the same as the page for the standard Disassociate transaction. The Disassociate (HPE) transaction removes child containers from parent containers. Child containers are typically disassociated from a parent when separate processing is required, for example when:

- A child container requires rework processing while the remaining child and parent containers do not.
- A child container requires processing through an alternate path.
- A child container must be placed on hold at a step.
- Several child containers require expedited processing to meet customer demand.

Refer to the *Opcenter Execution Medical Device and Diagnostics Shop Floor User Guide* or the *Opcenter Execution Core Shop Floor User Guide* for information on Disassociate.

Note: Disassociate (HPE) does not support transaction reversal.

Technical Uniqueness in Disassociate (HPE)

Disassociate (HPE) converts the Designer logic to SQL query and uses the SQL query to perform validation and submit data.

SQL statements, CLFs DBDisassociate_Initialize, DBDisassociate_Validate and DBDisassociate_Execute are created specifically for the Disassociate (HPE) transaction under the High Performance CLF in the Designer.

Elements in Disassociate (HPE)

CDO

A new CDO called DBDisassociate is created as a subclass of the Disassociate CDO. DBDisassociate_Initialize, DBDisassociate_Validate and DBDisassociate_Execute are attached to the Events.

Fields

Fields related to DBDisassociate are as follows:

- ChildContainers list object is inherited from Disassociate CDO.
- ContainerNamesStr is created and it is in the form of long string, containing all the child containers' names. The names of the child containers are separated by the delimiter "|".
- IsDBTxn is a flag that determines if the service belongs to HPE transaction. It is inherited from ContainerTxn CDO.

CLFs

- DBDisassociate_Execute - Contains all the INSERT/UPDATE/DELETE statements that were previously created within application server.
- DBDisassociate_Initialize - Creates temporary tables in the database. This CLF is used to store necessary data to be used as parameter for INSERT/UPDATE/DELETE statements in DBDisassociate_Validate and DBDisassociate_Execute and other DBCLF.
- DBDisassociate_Validate - Performs validation for container and child containers.

Start Two-Level (HPE)

The page for the Start Two-Level (HPE) transaction is the same as the page for the standard Start Two-Level transaction. This page allows you to start two different levels of containers. Quantities of single-level containers are defined, while quantities of multi-level containers are derived (or rolled up) from the quantities of the children containers. Container names must be unique regardless of container level.

Refer to the *Opcenter Execution Medical Device and Diagnostics Shop Floor User Guide* or the *Opcenter Execution Core Shop Floor User Guide* for information on Start Two-Level.

Note: Start Two-Level (HPE) does not support transaction reversal.

Technical Uniqueness in Start Two-Level (HPE)

SQL statements and CLF DBStart _Execute are created specifically for the Start Two-Level (HPE) transaction under the High Performance CLF in the Designer.

The value in the Child container grid cannot be edited as the container and Child container details follow the value in the parent container.

Elements in Start Two-Level (HPE)

CDO

A new CDO called DBStart is created as a subclass of the Start CDO. DBStart_Execute is attached to the Events.

Fields

Fields related to DBStart are as follows:

- ContainerNamesStr is created and it is in the form of long string, containing all the child containers' names. The names of the child containers are separated by the delimiter "|".
- IsDBTxn is a flag that determines if the service belongs to HPE transaction. It is inherited from ContainerTxn CDO.

CLF

- DBStart_Execute - Contains all the INSERT/UPDATE/DELETE statements that were previously created within application server.

Start Two-Level Simple (HPE)

The page for the Start Two-Level Simple (HPE) transaction is the same as the page for the standard Start Two-Level Simple transaction. This transaction creates parent and child container relationships. It enables you to group existing containers under a single parent. Then, an action performed once on the parent container applies to the child containers automatically.

Refer to the *Opcenter Execution Medical Device and Diagnostics Shop Floor User Guide* or the *Opcenter Execution Core Shop Floor User Guide* for information on Start Two-Level.

Note: Start Two-Level Simple (HPE) does not support transaction reversal.

Technical Uniqueness in Start Two-Level Simple (HPE)

SQL statements and CLF DBStart _Execute are created specifically for the Start Two-Level (HPE) transaction under the High Performance CLF in the Designer.

The value in the Child Container grid cannot be edited as the container and Child container details follow the value in the parent container.

SIEMENS

Start - Two Level Simple (HPE)

Container Details

Container *

Level *

Owner *

☐ Generate Names Automatically

Start Reason *

Mfg Order

Product

Workflow

Priority Code

Due Date

Expiration Date

Show additional fields (Container details)

Child Container Info

Level

Qty

UOM

Child Count

Add

☐ Generate Child Names Automatically

Show additional fields (Child Container details)

CHILD CONTAINERS

Container *

Level

Qty

UOM

Qty2

UOM2

No records to view

Show additional fields (Comments)

Reset

Start

Elements in Start Two-Level Simple (HPE)

CDO

A new CDO called DBStartSimple is created as a subclass of the DBStart CDO. DBStart_Initialize and DBStart_Execute are attached to the Events.

Fields

Fields related to DBStartSimple are as follows:

- ContainerNamesStr is created and it is in the form of long string, containing all the child containers' names. The names of the child containers are separated by the delimiter "|".
- IsDBTxn is a flag that determines if the service belongs to HPE transaction. It is inherited from ContainerTxn CDO.

CLF

- DBStart_ExecuteEx - Contains all the INSERT/UPDATE/DELETE statements that were previously created within application server.
- DBStart_Initialize - Creates temporary tables in the database and this CLF is used to store necessary data to be used as parameter for INSERT/UPDATE/DELETE statements in DBStart_ExecuteEx and other DBCLF.

Terminate Lot (HPE)

Terminate Lot (HPE) transaction works the same as the standard Lot Terminate and Multi-Lots transactions. The transaction is used to terminate one or multiple lots.

Refer to the *Opcenter Execution Semiconductor Shop Floor User Guide* for information on Lot Terminate or Multi-Lots Terminate.

Note: Terminate Lot (HPE) does not support transaction reversal.

Technical Uniqueness in Terminate Lot (HPE)

The standard Lot Terminate transaction converts Designer logic to SQL query and uses the SQL query to perform data validation and submission.

For Terminate Lot (HPE), SQL statements are created in Designer including its own CLFs. The CLFs are DBTerminateLot_Initialize and DBTerminateLot_Execute which are created specifically for the Terminate Lot HPE transaction under High Performance CLFs.

Elements in Terminate Lot (HPE)

CDO

A new CDO called DBTerminateLot is created as a subclass of Terminate Lot CDO. DBTerminateLot_Initialize and DBTerminateLot_Execute are attached at the Events.

Field

This field is related to DBTerminateLot:

- IsDBTxn is a flag that determines if the service belongs to HPE transaction. It is inherited from ContainerTxn CDO.

CLFs

- DBTerminateLot_Initialize - Creates temporary tables in the database and is used to store necessary data to be used as parameters for INSERT/UPDATE/DELETE statements in DBTerminateLot_Execute and other DBCLFs.
- DBTerminateLot_Execute - Contains all the INSERT/UPDATE/DELETE statements that were previously created within Application Server.

XML Sample

The child containers are using ContainerNamesStr list string to achieve the efficiency in speeding up the processes. This is applicable to all HPE transactions except for Start Two-Level (HPE).

```
<ContainerNamesStr><![CDATA[CNP0010 - 01|CNP0010 - 02|CNP0010 - 03|CNP0010 - 04|CNP0010 - 05]]></ContainerNamesStr>
```

Opcenter Execution Core Transaction Tester in HPE

For automation using Opcenter Execution Core Transaction Tester (CTT), the field for child container name submission is ContainerNamesStr. A long string can be submitted using the pipe character, "|". The delimiter is processed by the DBCLFFunctions script file.

For example: A|B|C|D|E.

Parameter Template Delimiter in CTT needs to change from * to |.

Opcenter Execution Core Transaction Tester - C:\HPE Installer\QESLoaderScript.cfg

File Reports Help

Test Details

Server Address: localhost Model ID: Client ID: Test ID: Run Iterations: 1

Begin Test Stop Test ☐ Stop On Error Errors: Success: ☐ Enable Prompt (all)

Script Configuration Global Parameters Script Settings Test Result Details Misc

Application Settings	
Username	CamstarAdmin
Password	*****
Host	localhost
Port	443
Auto TXNGUID	False
Auto Session Element	True
Global Parameters	(Collection)
Parameter Templates	(Collection)
Parameter Template Delimiter	*
Transaction Templates	(Collection)
XPath Hints	(Collection)
CSV Delimiter	.
Misc	
LogFileFolder	C:\Temp
DefaultPluginName	Submit To Server
DefaultPluginSettings	(Collection)
Test Settings	
Continuous Run	False
Continuous Run CSV Filename	
Run Iterations	1
Stop on Error	False
Force UTF16LE	False

WIP Messages

WIP Messages in HPE work the same as in the standard transaction pages. DBCLF_ProcessWIPMsgs in HPE mimic the original WIP Messages process. This CLF is attached to the ContainerTxn_BeforeExecute.

There are three sections of HPE Process WIP Msgs logic:

- Send Notification – Send email notification when WIP Msg is submitted.
- Stop Processing – Put the container on hold after WIP Msg is submitted.
- Write To History – Insert a history record to WIPMsgHistoryDetail table.

Send Notification still uses the Application Server approach as the non-HPE pages do. DBCLF_ProcessWIPMsgs calls the original SendNotification event from WIP Msg when the conditions are met.

Stop Processing is converted to HPE. When stop processing criteria is met, the system will:

- UPDATE the container table with hold information,
- INSERT a hold record to History Mainline table,
- INSERT a History Mainline Details record, and
- INSERT a record in Hold Release History table.

Write To History is converted to HPE. When *Write To History* is enabled in the WIP Msg, the system will INSERT a new record into the WIPMsgHistoryDetail table.

E-Signature

The basic processes are similar to the standard shop floor transactions.

In HPE, the CLF `DBCLF_ProcessElectronicSignatures` contains the execution of SQL statements. The SQL statements insert the history details of E-Signature. This CLF is attached at `ContainerTxn CDO`. The CLF will only execute on the HPE shop floor transaction if there are E-Signatures configured on the modeling object and it is a common function for HPE transaction. `DBCLF_ProcessElectronicSignatures` is processed after `DBCLF_Initialized`, `DBCLF_Validate`, and `DBCLF_Execute` are called. `DBCLF::HistoryMainlineId` parameter in the `DBCLF_Execute` is required to run the `DBCLF_ProcessElectronicSignatures`.

Process Timers

Process Timers are available with Associate (HPE) and Disassociate (HPE). Associate (HPE) and Disassociate (HPE) have been updated with the Process Timers capability like the standard versions of these transactions.

Associate (HPE) can delete the child containers' timer records from the database. The records are not needed anymore because the child containers will use the same timer as the parent container after association.

Disassociate (HPE) can copy the parent's timer records to the child containers. Each individual child container will have its own timer after disassociation.

DBCLF_ProcessTimers event is added to the ContainerTxn object in Designer. This event is overwritten in DBAssociate and DBDisassociate because each of them has unique execution. DBAssociate deletes records while DBDisassociate creates new records. DBCLFProcessTimers is executed after DBCLF_ProcessWIPMsgs.

Numbering Rule

Start Two-Level Simple (HPE) makes use of the new Numbering Rule function to generate the child container names.

A new method, DBGenerateSequence is added to the Numbering Rule CDO to generate numbering sequences in bulk. This method can be called by any other service. Calling the method will execute the logic which then stores the generated sequences in a DBCLF variable.

The input for the DBGenerateSequence method are as follows:

- FieldLengthLimit: Limit of the field length. Optional and can be left null
- SequencesRequested: Number of sequences to generate
- DBCLFVariableName: Name of the DBCLF variable that will store the generated sequences in the DB.

The generated sequences are set as a string (stored in the DBCLF variable) in the following delimited format:

`<sequence01>|<sequence02>|sequence<03>|...`

Chapter 5: Debugging High Performance CLF Transactions

Introduction

You can troubleshoot and debug High Performance CLF transaction logic using an error log and a trace log.

In This Chapter

This chapter contains this topic:

- Error Logging and Trace Logging

Error Logging and Trace Logging

Error logging and trace logging facilitate development, debugging, and troubleshooting when processing High Performance transactions. There are utility stored procedures to log the errors and tracing. All utility stored procedures contain the **clfutil** prefix.

Errors

The application writes any errors it encounters when processing a High Performance CLF to this table along with the timestamp and JSON document:

CLFErrorLog

The utility stored procedure that logs the errors and JSON document to the **CLFErrorLog** table follows:

clfutilLogError()

The application writes to this table even if tracing is turned off by setting the tracing level to 0. Refer to "Tracing."

Tracing

The application writes tracing (non-error) information to this table:

CLFTraceLog

You can configure multiple levels of tracing by using the InSiteSiteInfo table parameter named **DBCLFTraceLevel**. A numeric level determines how much detail to trace.

This table defines each trace level and what information is traced.

Trace Level	Definition	Information Traced
0	No tracing	Nothing
1	Basic tracing	SQL text, rows processed, CLF response and CLF result (success/failure)
2	Enhanced tracing	Parameter values

The utility stored procedure that handles tracing and logging of messages, and writes to the **CLFTraceLog** table if the trace level is higher than or equal to the trace level set in the InSiteSiteInfo table parameter follows:

clfutilLogTrace()

Example:

You can issue a statement such as the following using a database tool to change the trace level:

```
UPDATE InSiteSiteInfo SET TValue='2' WHERE TName='DBCLFTraceLevel'
```

Viewing Trace Logs

You can view a trace log by completing the following:

- Turn on tracing (specify DBCLFTraceLevel > 0)
- Execute the transaction
- Query the CLFTraceLog table with a query such as:
SELECT * FROM CLFTraceLog ORDER BY LogDate

The SELECT statement returns all trace information in the table.

This table describes the information contained in the columns.

Column	Description
LogDate	Timestamp of the trace.
CLFID	Unique ID for the High Performance CLF constructed from the TxnID and the CLF Name.
LogMessage	Trace message.

This image shows an example of the columns in a trace log.

LOGDATE	CLFID	LOGMESSAGE
23-OCT-20 10.31.35.494000000	AM 4000000000001368-BeforeExecute	BEGIN
23-OCT-20 10.31.35.496000000	AM 4000000000001368-BeforeExecute	{"Functions":[{"Name":"DBValidateNotExists","ID":"3835","Parameters":[{"Name":"ErrorMessage":
23-OCT-20 10.31.35.567000000	AM 4000000000001368-BeforeExecute	SQL: SELECT 'X' FROM CONTAINER WHERE ContainerName = ?ContainerName
23-OCT-20 10.31.35.568000000	AM 4000000000001368-BeforeExecute	CONTAINERNAME=DEMI_C1_J3_1
23-OCT-20 10.31.35.569000000	AM 4000000000001368-BeforeExecute	Rows processed: 0
23-OCT-20 10.31.35.616000000	AM 4000000000001368-BeforeExecute	SQL: INSERT INTO Container(ContainerID,ContainerName,LevelId,OriginalFactoryId,CDOTypeID,C
23-OCT-20 10.31.35.622000000	AM 4000000000001368-BeforeExecute	BOMID=0018a60000000005
23-OCT-20 10.31.35.622000000	AM 4000000000001368-BeforeExecute	CURRENTSTATUSID=00085c8000000e07
23-OCT-20 10.31.35.622000000	AM 4000000000001368-BeforeExecute	CONTAINERNAME=DEMI_C1_J3_1
23-OCT-20 10.31.35.622000000	AM 4000000000001368-BeforeExecute	CONTAINERID=0004108000007fb0
23-OCT-20 10.31.35.623000000	AM 4000000000001368-BeforeExecute	LEVELID=0005280000000004
23-OCT-20 10.31.35.623000000	AM 4000000000001368-BeforeExecute	OWNERID=0005460000000002
23-OCT-20 10.31.35.623000000	AM 4000000000001368-BeforeExecute	ORIGINALFACTORYID=0004e28000000003
23-OCT-20 10.31.35.623000000	AM 4000000000001368-BeforeExecute	MFGORDERID=0006548000000002
23-OCT-20 10.31.35.624000000	AM 4000000000001368-BeforeExecute	PARENTCONTAINERID=
23-OCT-20 10.31.35.624000000	AM 4000000000001368-BeforeExecute	STARTDATEGMT=10/23/2020 2:31:35 PM
23-OCT-20 10.31.35.624000000	AM 4000000000001368-BeforeExecute	STARTDATE=10/23/2020 10:31:35 AM
23-OCT-20 10.31.35.624000000	AM 4000000000001368-BeforeExecute	PRODUCTID=00062c000000000b
23-OCT-20 10.31.35.625000000	AM 4000000000001368-BeforeExecute	STARTREASCHID=0007800000000002
23-OCT-20 10.31.35.625000000	AM 4000000000001368-BeforeExecute	UCHID=0008de8000000003
23-OCT-20 10.31.35.625000000	AM 4000000000001368-BeforeExecute	UCH2ID=
23-OCT-20 10.31.35.626000000	AM 4000000000001368-BeforeExecute	Rows processed: 1

This table describes the information contained in the columns if a full trace (DBCLFTraceLevel = 2) is executed.

Column	Description
BEGIN/END	First and last rows written to the trace table for the given High Performance CLF.
JSON package	Can be used to replay the DB CLF. Refer to "Replaying a High Performance CLF."
SQL statement	Raw SQL statement.

Column	Description
Parameters and values	Parameters and values for the preceding SQL statement.
Rows processed	Number of rows affected by the preceding SQL statement.
Error message	Full error message along with location in the logic in which the error occurred. This same message is written to the CLFErrorLog table even if tracing is turned off.

Replaying a High Performance CLF

You can replay a High Performance CLF by copying the High Performance CLF JSON package and sending that package to the `clfExecute` stored procedure (the main CLF stored procedure). You can also edit the JSON package before sending it to the `clfExecute` stored procedure.

How to Replay a High Performance CLF:

Follow these steps to replay a High Performance CLF:

1. Query either the `CLFErrorLog` or the `CLFTraceLog` table and find the row that contains the JSON package for the DBCLF you want to replay.
2. Copy the full string from that row.

This image shows an example of the full string that can be copied.

LOGDATE	CLFID	LOGMESSAGE
23-OCT-20 10.31.35.494000000	AM/400000000001340-BeforeExecute	BEFORE
23-OCT-20 10.31.35.494000000	AM/400000000001340-BeforeExecute	["Functions":[{"Name":"DBValidateNotExists","ID":"3835","Parameters":{"Name":"ErrorMessages","Value":"Container DBM_C1_33_1 already exists
23-OCT-20 10.31.35.547000000	AM/400000000001340-BeforeExecute	SQL: SELECT 'X' FROM CONTAINER WHERE ContainerName = 'ContainerName
23-OCT-20 10.31.35.548000000	AM/400000000001340-BeforeExecute	CONTAINERNAME=CONTAINER_C1_33_1
23-OCT-20 10.31.35.548000000	AM/400000000001340-BeforeExecute	Rows processed: 0
23-OCT-20 10.31.35.614000000	AM/400000000001340-BeforeExecute	SQL: TRUNC INTO Container (ContainerID, ContainerName, LevelID, OriginalFactoryID, CDTypeID, ChangeCount, ChildCount, FactoryStartDate, FactoryS...
23-OCT-20 10.31.35.622000000	AM/400000000001340-BeforeExecute	SQLID=0014e0000000005
23-OCT-20 10.31.35.622000000	AM/400000000001340-BeforeExecute	CURRENTSTATUSID=0008c0000000e7
23-OCT-20 10.31.35.622000000	AM/400000000001340-BeforeExecute	CONTAINERNAME=CONTAINER_C1_33_1
23-OCT-20 10.31.35.622000000	AM/400000000001340-BeforeExecute	CONTAINERID=0004100000007260
23-OCT-20 10.31.35.622000000	AM/400000000001340-BeforeExecute	LEVELID=0005200000000004
23-OCT-20 10.31.35.622000000	AM/400000000001340-BeforeExecute	SQLID=0005400000000002
23-OCT-20 10.31.35.622000000	AM/400000000001340-BeforeExecute	ORIGINALFACTORYID=0004e2000000003
23-OCT-20 10.31.35.622000000	AM/400000000001340-BeforeExecute	ROWNUMBER=0004e2000000003

3. Call the `clfExecute` stored procedure using the following PL/SQL block in Oracle:

```

DECLARE
  pRet NUMBER;
  pResp CLOB;

  -- UC04: CLF parameters
  vJSON CLOB := '{"Functions":[{"Name":"DBValidateNotExists","ID":"3835","Parameters":{"Name":

BEGIN
  clfExecute('123456',vJSON,pRet,pResp);

  DBMS_OUTPUT.PUT_LINE('Response: '||pResp);
END;
```

Note: The `vJSON CLOB` should contain the JSON package that was copied from the trace log or error log table.

Clearing the Error and Trace Logs

You can clear the logs by issuing a TRUNCATE SQL statement.

How to Clear the Trace and Error Logs

Enter these statements to clear the logs:

```
TRUNCATE TABLE CLFTraceLog;
```

```
TRUNCATE TABLE CLFErrorLog;
```


Index

A

architecture 1-2

Associate (HPE) 4-2

C

clearing logs 5-5

D

database functions 2-4

DBService 3-3

DBStart 3-3, 3-6

debugging 5-2

Disassociate (HPE) 4-3

E

E-Signature 4-11

error log

- clearing 5-5

error logging 5-2

example of a transaction 3-2

H

High Performance

- CLFs 1-5
- overview 1-1, 1-4
- transactions 1-5

J

JSON document example 2-2

M

main logic 3-6

N

Numbering Rule 4-13

P

process timers 4-12

processing HPE CLFs 2-2

R

replay

- High Performance CLF 5-4

S

sample CDOs 3-3

sample service

- creating 3-3

sample transaction 1-3, 3-2

security information ix

SQL statements 1-2

- types 1-4

Start Two-Level (HPE) 4-4

stored procedures

- main CLF 2-3
- supporting 2-4

T

Terminate Lot (HPE) 4-7

trace log

- clearing 5-5
- viewing 5-3

trace logging 5-2

Transaction Tester 4-9

troubleshooting 5-2

W

warning

- sample transaction 3-2

WIP Messages 4-10

X

XML sample 4-8

Siemens Digital Industries Software

Headquarters

Granite Park One
5800 Granite Parkway
Suite 600
Plano, TX 75024
USA
+1 972 987 3000

Office(s)

13024 Ballantyne Corporate Place
Suite 300
Charlotte, NC 28277
USA
+1 704 227 6600

About Siemens Digital Industries Software

Siemens Digital Industries Software is driving transformation to enable a digital enterprise where engineering, manufacturing and electronics design meet tomorrow. Our solutions help companies of all sizes create and leverage digital twins that provide organizations with new insights, opportunities and levels of automation to drive innovation. For more information on Siemens Digital Industries Software products and services, visit siemens.com/software or follow us on [LinkedIn](#), [Twitter](#), [Facebook](#) and [Instagram](#). Siemens Digital Industries Software – Where today meets tomorrow.

© Siemens 2021

A list of relevant Siemens trademarks can be found [here](#).
Other trademarks belong to their respective owners.